

It is completely normal to feel a bit unsure when translating a conceptual academic plan into concrete code. Transitioning your research from unsupervised learning (clustering) to supervised learning (regression) is a fantastic extension of your thesis, and it builds perfectly on the rigorous pipeline you have already established!

Based on the notes provided by your advisor (or yourself), your plan is highly structured and very feasible. Let's ground this in reality by breaking down exactly how you will implement each of the four steps in R.

1. Generating Original Data (OD)

You need to simulate a multivariate normal distribution for your predictors and then calculate the response variable y using a linear equation.

- **The Predictors (X):** You will use the `rmvnorm` function. *Note: You have a slight typo in your notes; the package is `mvtnorm`, which you actually used successfully in your original clustering research.* To create the covariance matrix Σ_1 with 1 on the diagonal and ρ on the off-diagonals, you can generate a matrix of ρ values and then replace the diagonal with 1.
- **The Coefficients (β):** You will define a vector where $\beta_0, \beta_1, \dots, \beta_{10}$ all equal 1.
- **The Response (y):** You will calculate y using matrix multiplication of your predictors and coefficients, plus the normally distributed error term $\epsilon \sim N(0, \sigma_2)$ where $\sigma_2 = 1$.

2. Generating Synthetic Data (SD)

For this step, you will stick to the methodology proven in your thesis.

•

The Method: You will use the Classification and Regression Trees (CART) method via the `synthpop` R package.

- **Implementation:** Once your Original Data is in a data frame, you will simply pass it through the `syn()` function, specifying `method = "cart"`.

3. Regression Analysis on OD and SD

This is the most straightforward step. You will fit a standard linear model on both datasets to see how the CART synthesis alters the linear relationships.

- **Implementation:** You will use the standard `lm()` function.
- Original Model: `lm_OD <- lm(y ~ ., data = OD_dataframe)`
- Synthetic Model: `lm_SD <- lm(y ~ ., data = SD_dataframe)`

4. Similarity Measures (The Evaluation)

Just as you used structural metrics like the Mean Centroid Distance to measure clustering degradation, you need mathematical ways to measure regression degradation.

- **Euclidean Distance between Coefficients:** Extract the estimated $\hat{\beta}$ vectors from both `lm_OD` and `lm_SD`. Calculate the distance using the formula:

$$d(\hat{\beta}_{OD}, \hat{\beta}_{SD}) = \sqrt{\sum_{j=0}^p (\hat{\beta}_{OD, j} - \hat{\beta}_{SD, j})^2}$$

- **Distance between Predictions:** Generate a brand new, unseen "test" dataset using the exact same logic as Step 1. Then, use both `predict(lm_OD, newdata = test_data)` and `predict(lm_SD, newdata = test_data)`. You can calculate the Mean Squared Error (MSE) or Mean Absolute Error (MAE) between these two sets of predictions to see how much the synthetic model's predictions drift from the original model's predictions.

This structured approach perfectly mirrors the reverse-engineering pipeline you built for your clustering thesis, just applied to regression.

Would you like me to draft the actual R code template for this pipeline so you can copy it directly into RStudio and start experimenting?